

你的算力用的好么——推理集群优化的三个阶段

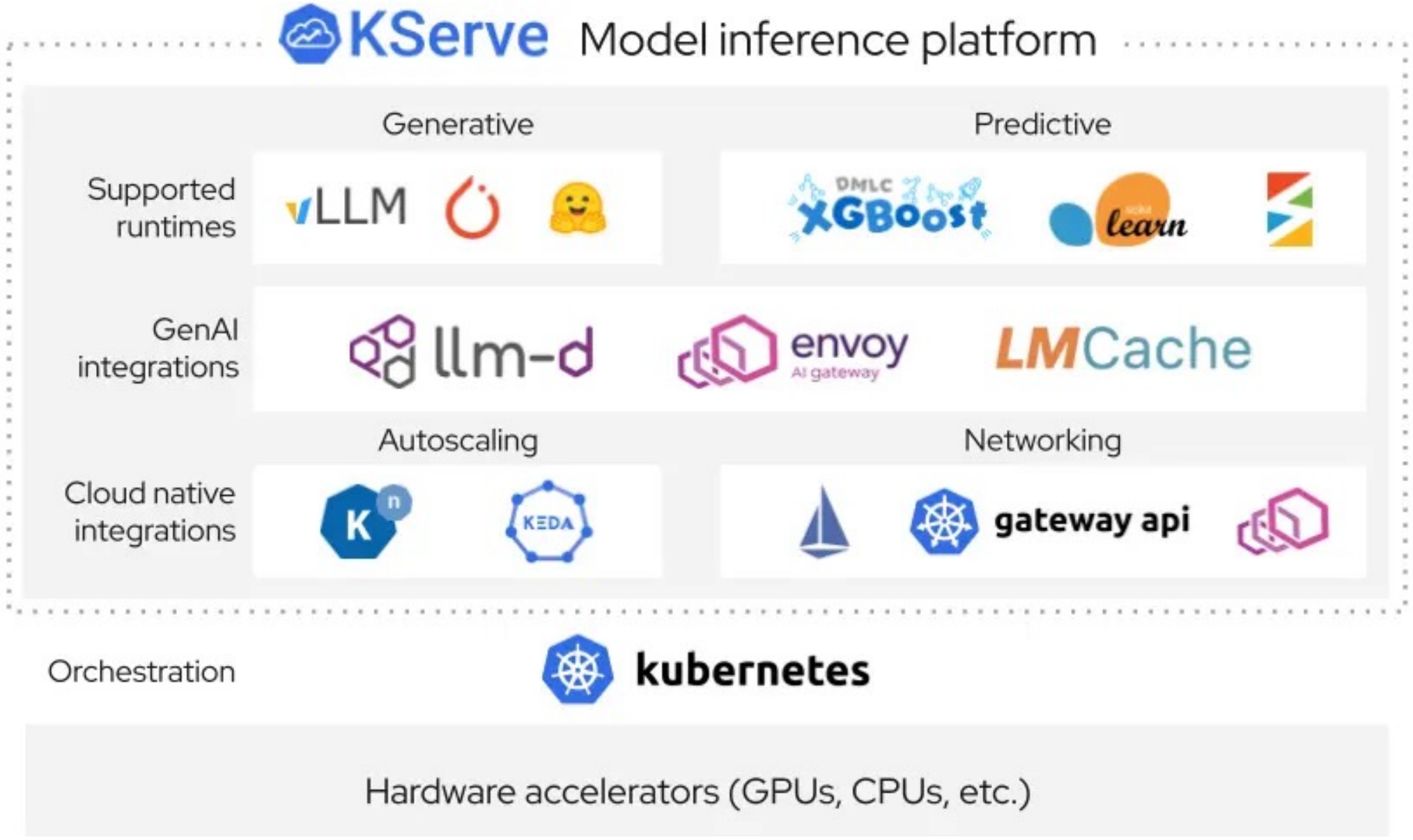
李孟轩

DYNAMIA.AI



- Phase 1: Inference as a workload
- Phase 2: Inference + kvcache/PD分离
- Phase 3: Inference + Kvcache + PD分离 + GPU虚拟化

AI Infra的实践：RedHat KServe推理平台



AI Infra解决的问题



弹性伸缩

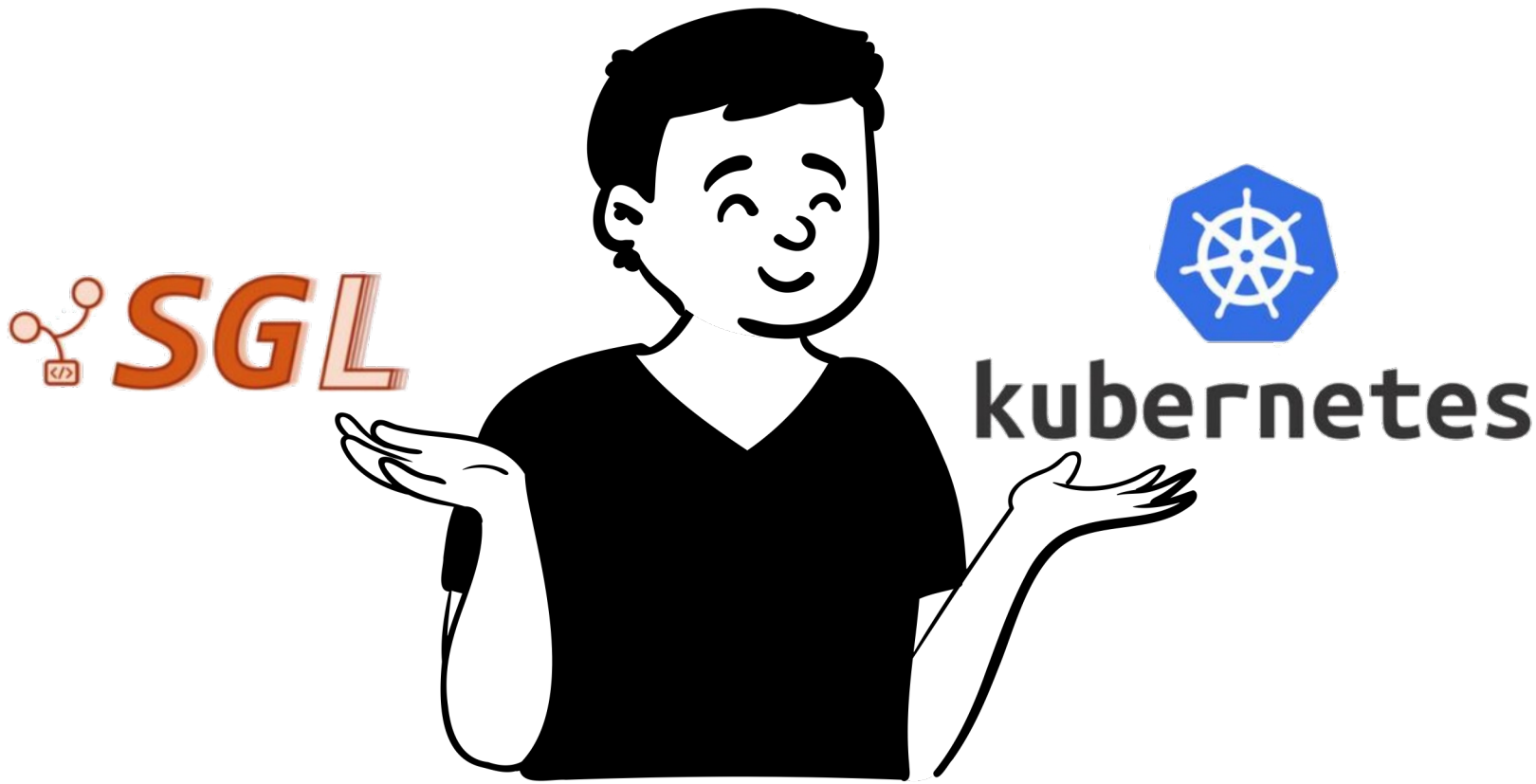


负载均衡

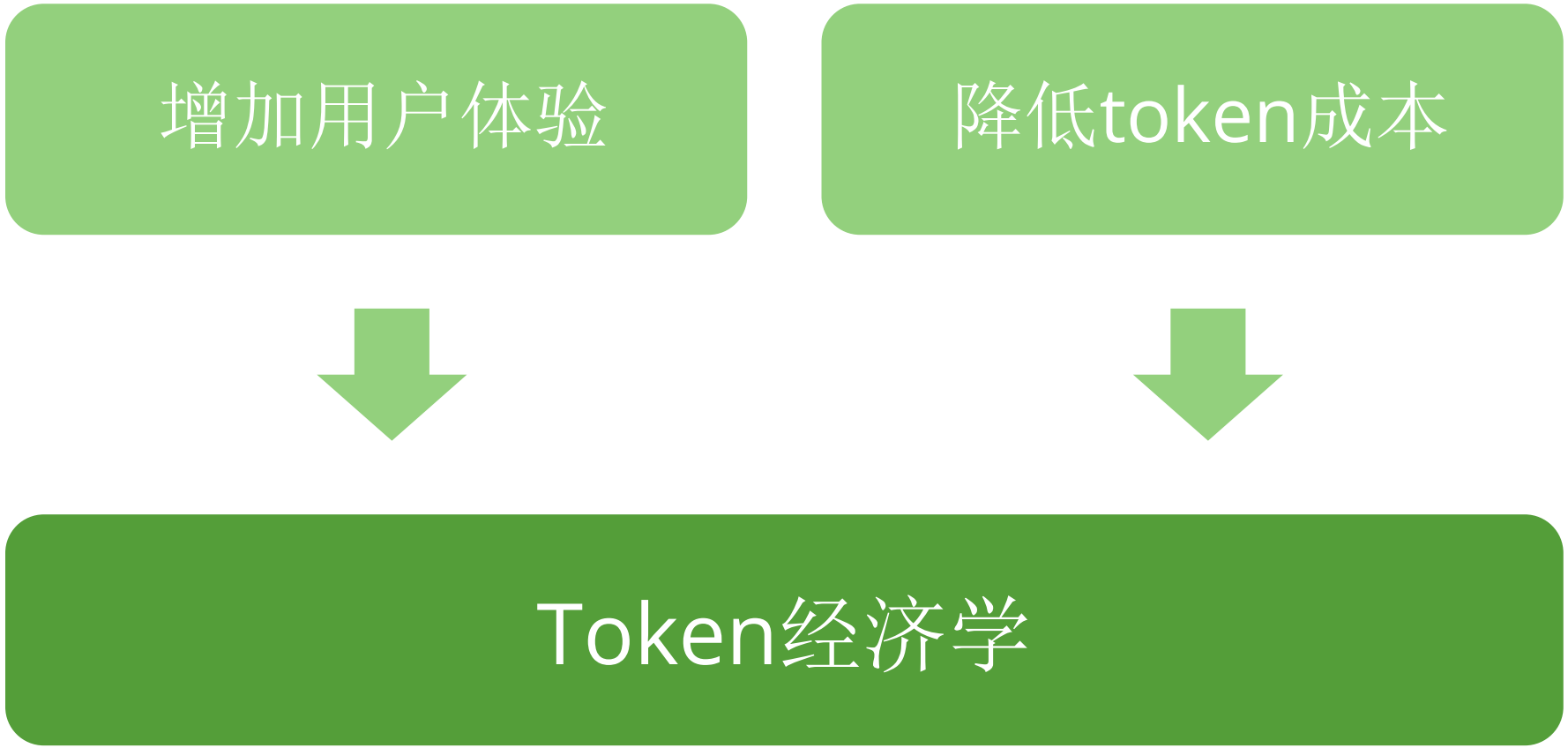


系统/服务的稳定性

2个基础组件足以解决上述问题



Agentic AI时代下的AI Infra



以进程的形式运行SGL

你有一个GPU服务器,

你希望用最快的速度验证vLLM的推理



> vLLM serve qwen3-32b



以其他形式运行vLLM

以虚拟环境的形式运行vLLM

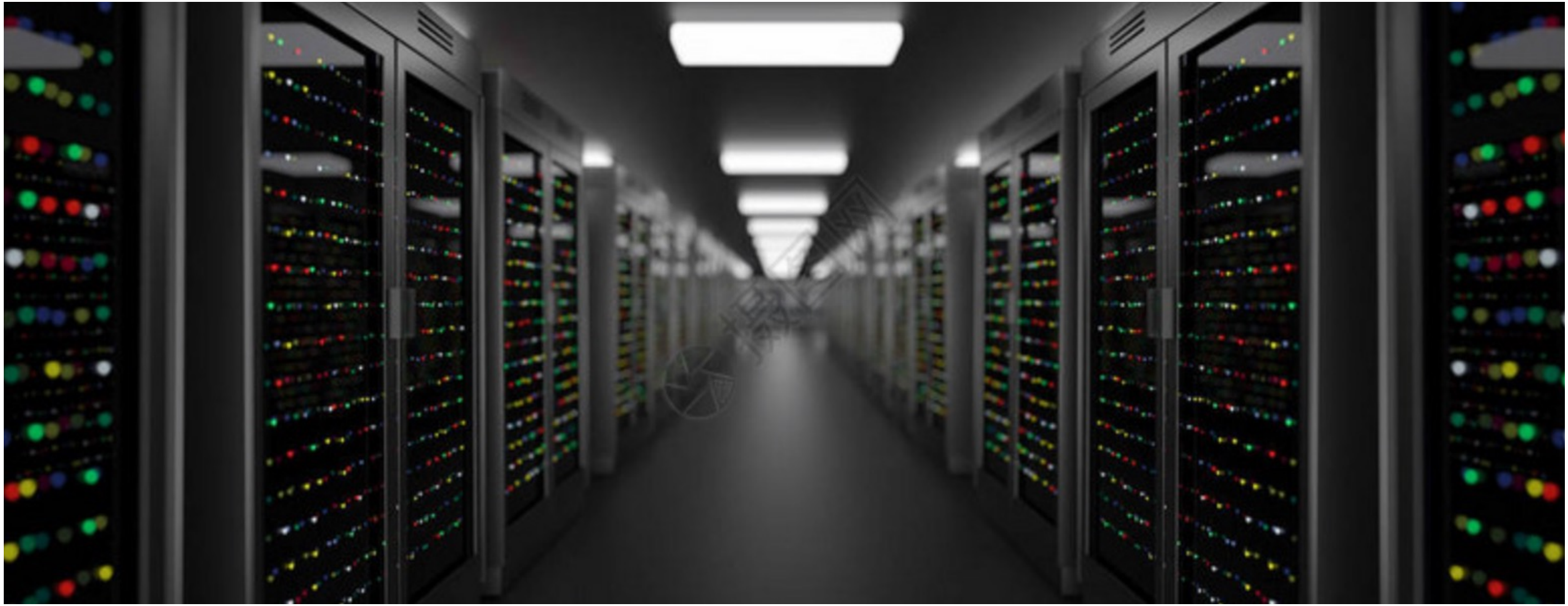
```
conda create -n sglang python=3.11 -y
conda activate sglang
pip install --upgrade pip
pip install uv
uv pip install --prerelease=allow sglang
CUDA_VISIBLE_DEVICES=0,1,2,3 python -m
sglang.launch_server \
  --model-path meta-llama/Meta-Llama-3-70B-
Instruct \
  --host 0.0.0.0 \
  --port 8000 \
  --tensor-parallel-size 4 \
  --mem-fraction-static 0.9
```

以容器的形式运行vLLM

```
docker run --runtime nvidia --gpus
'"device=0,1,2,3"' \
  -v
~/.cache/huggingface:/root/.cache/huggingface \
  -p 8000:8000 \
  --ipc=host \
  --shm-size 32g \
  lmsysorg/sglang:latest \
  python3 -m sglang.launch_server \
    --model-path meta-llama/Meta-Llama-3-8B-
Instruct \
    --host 0.0.0.0 \
    --port 8000 \
    --tensor-parallel-size 1 \
    --mem-fraction-static 0.9
```

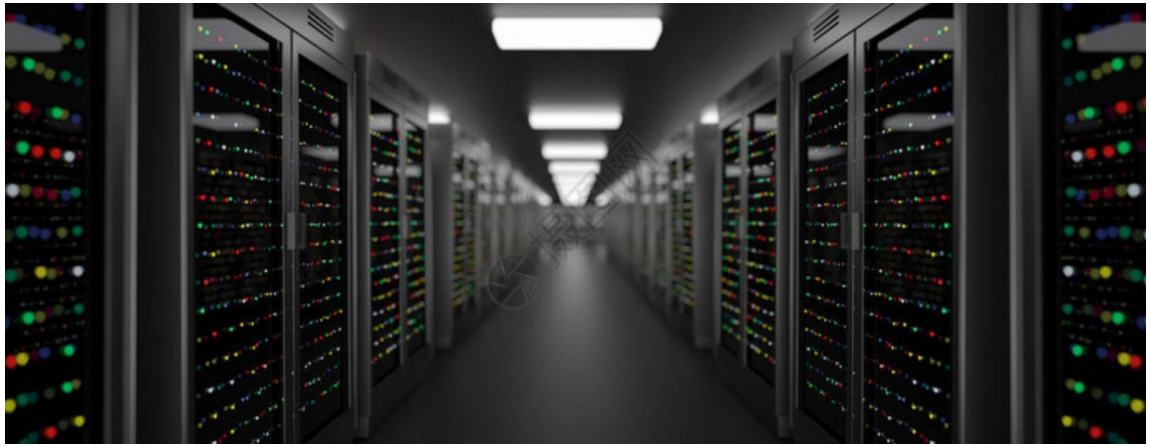
Phase 1: inference as a workload

适用场景：你希望在一个GPU集群中分配vLLM工作负载



在GPU集群中分配vLLM工作负载

kubernetes → SGL →

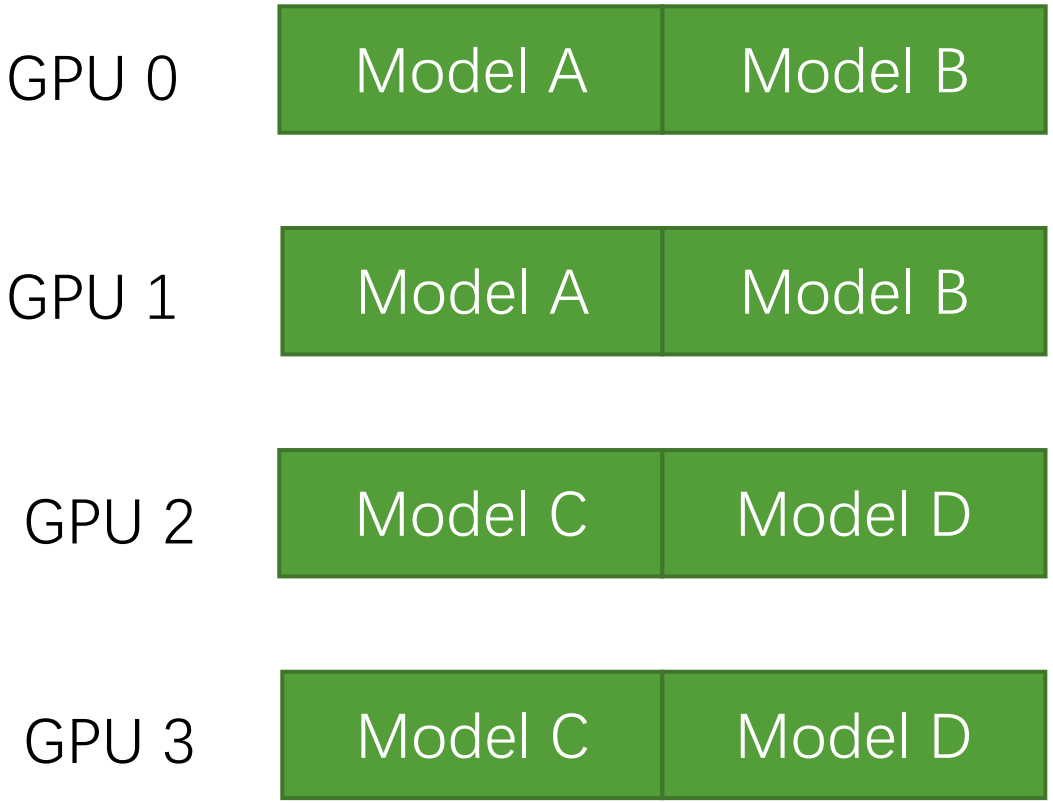


```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sglang-server
spec:
  replicas: 1
  template:
    spec:
      containers:
      - name: sglang
        image: lmsysorg/sglang:latest
        command: ["python3", "-m",
"sglang.launch_server"]
        args:
        - --model-path
        - meta-llama/Meta-Llama-3-8B-
Instruct
        - --mem-fraction-static
        - "0.9"
        - --context-length
        - "8192"
        ports:
        - containerPort: 8000
        resources:
          limits:
            nvidia.com/gpu: 1
            memory: "32Gi"
            cpu: "8"
        ...
```

Test Case: HAMi-core VS Mig

CLOUD NATIVE AI + INFERENCE DAY NORTH AMERICA

4 TP=2 Qwen3-8B using GPU slicing：
GPU0, GPU1 using MIG(3g.20gb)
GPU2,GPU3 using HAMi-core(nvidia.com/gpumem=20g)



low cache hit~13%

QPS	HAMi-core tok/s	MIG tok/s	HAMi-core TTFT p90	MIG TTFT p90
0.5	48.1	31.8	1.57s	1.39s
1	94.3	81.6	1.84s	1.84s
2	201.1	184.7	2.3s	5.97s
3	214.3	179.2	8.01s	24.3s

high cache hit~82%

QPS	HAMi-core tok/s	MIG tok/s	DRA TTFT p90	MIG TTFT p90
1	173	207	1.55s	1.87s
2	242	368	1.51s	2.87s
3	486	607	1.50s	1.82s
6	606	474	7.28s	28.0s
10	844	793	14.9s	30.4s

Phase 2: 推理框架 + PD分离

Hybrid GPU:



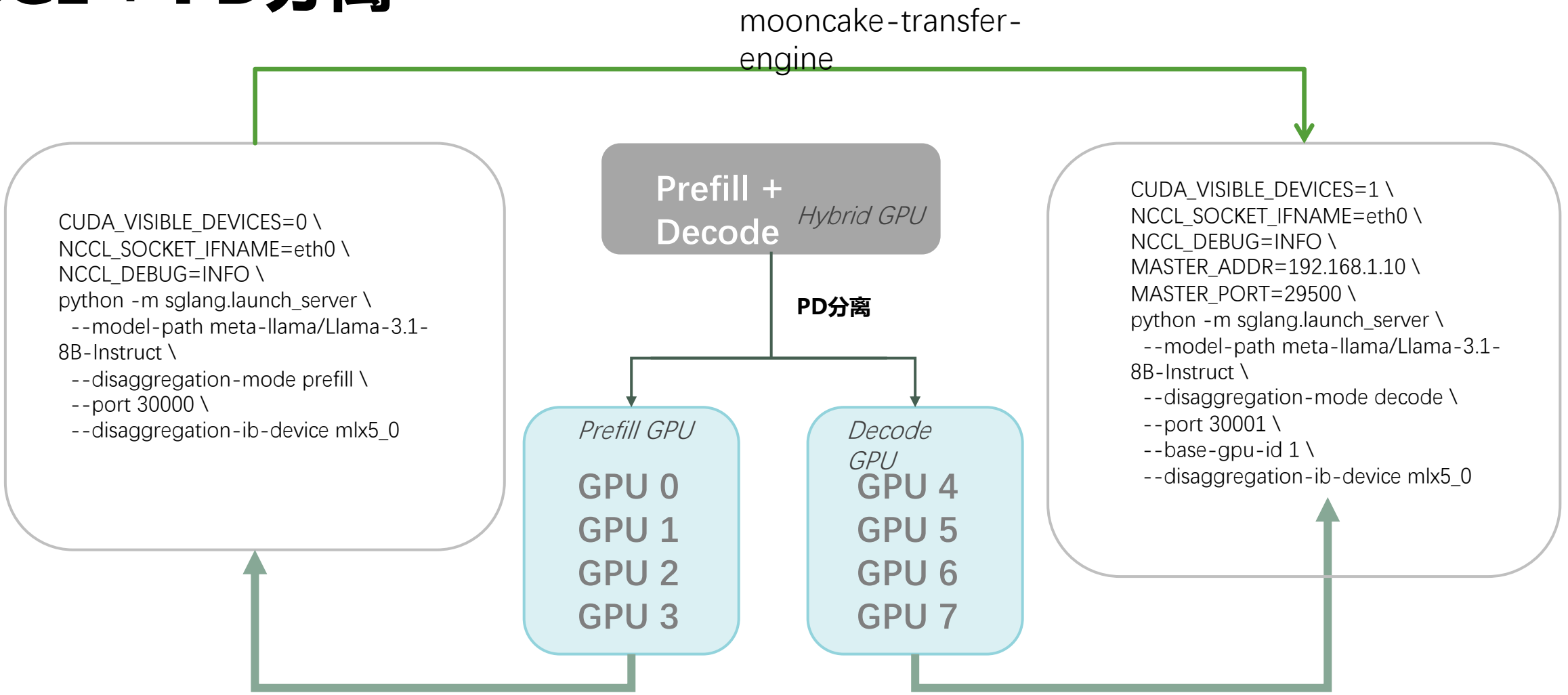
Prefill GPU



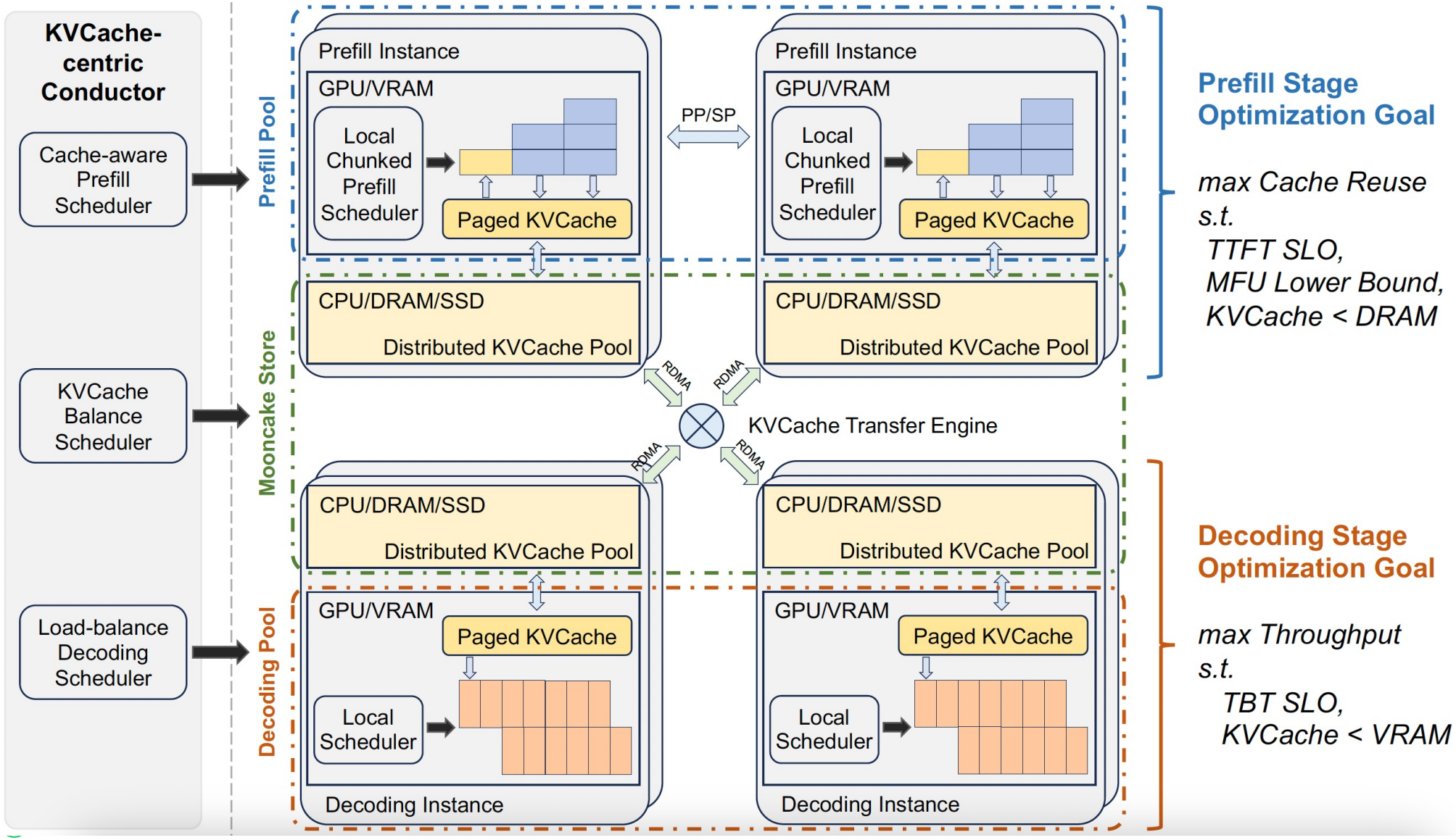
Decode GPU



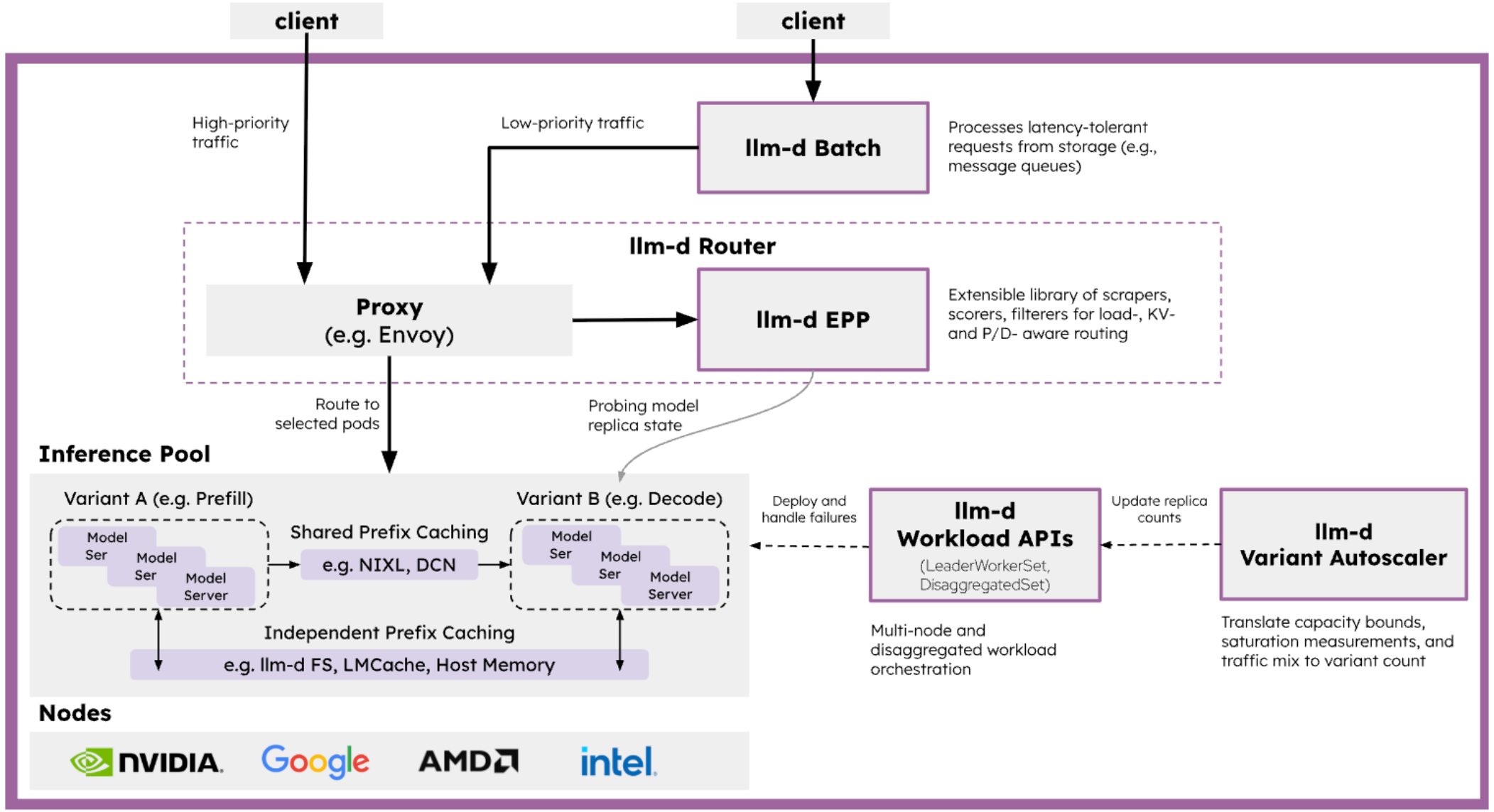
SGL + PD分离


mooncake-transfer-engine
CUDA_VISIBLE_DEVICES=0 \
NCCL_SOCKET_IFNAME=eth0 \
NCCL_DEBUG=INFO \
python -m sglang.launch_server \
--model-path meta-llama/Llama-3.1-8B-Instruct \
--disaggregation-mode prefill \
--port 30000 \
--disaggregation-ib-device mlx5_0
Prefill + Decode
Hybrid GPU
PD分离
Prefill GPU
GPU 0
GPU 1
GPU 2
GPU 3
Decode GPU
GPU 4
GPU 5
GPU 6
GPU 7
CUDA_VISIBLE_DEVICES=1 \
NCCL_SOCKET_IFNAME=eth0 \
NCCL_DEBUG=INFO \
MASTER_ADDR=192.168.1.10 \
MASTER_PORT=29500 \
python -m sglang.launch_server \
--model-path meta-llama/Llama-3.1-8B-Instruct \
--disaggregation-mode decode \
--port 30001 \
--base-gpu-id 1 \
--disaggregation-ib-device mlx5_0

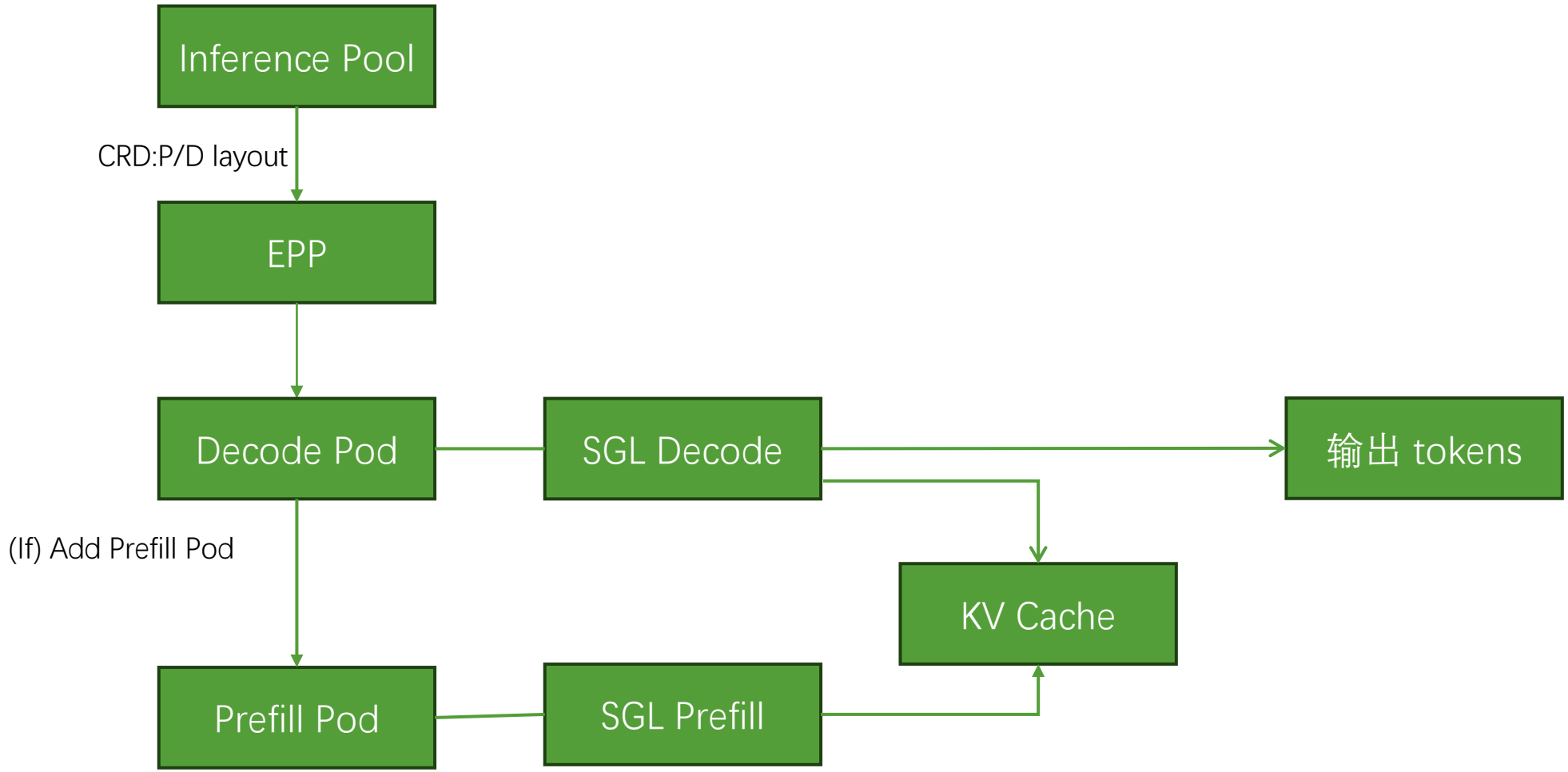
Phase 2: SGL + PD + Routing: Mooncake



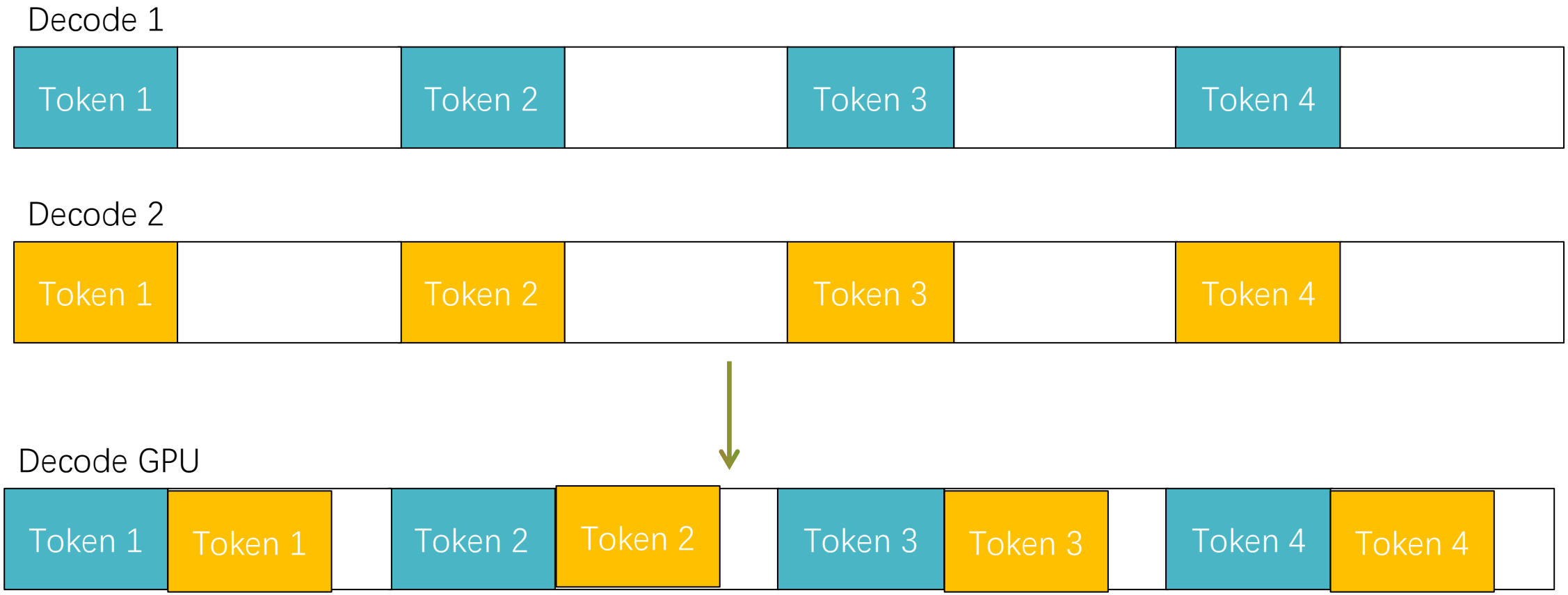
Phase 2: SGL + PD + Routing： LLM-D



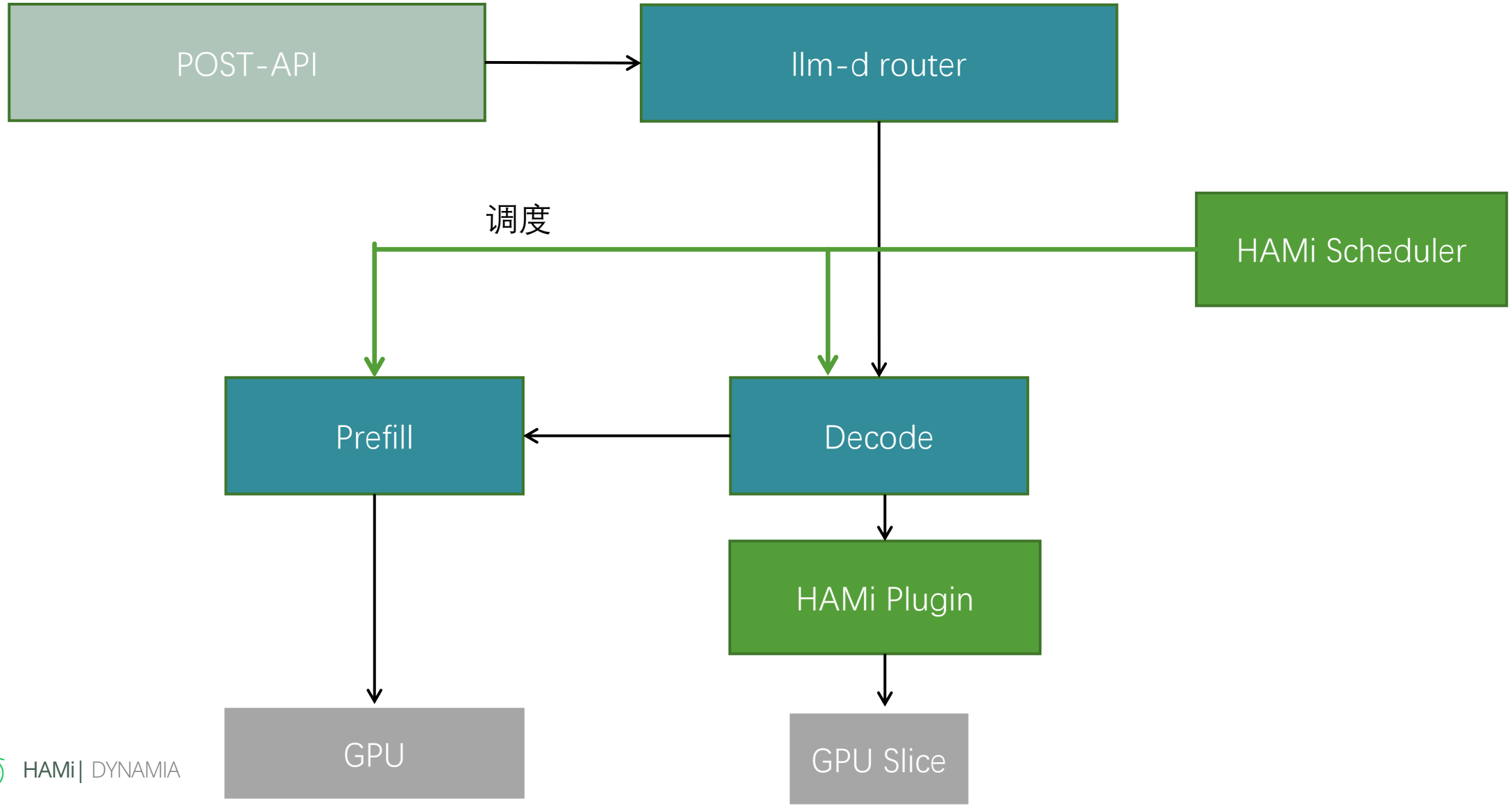
sgl + PD分离 + Routing——LLM-D



Phase 3: SGL + PD分离 + GPU虚拟化



Phase 3: SGL + PD分离 + HAMi



整体架构


模型层
Qwen
deepseek
KIMI
推理引擎
vLLM
SGL
推理框架
llm-d
NVIDIA Dynamo
Mooncake
调度
HAMi
VOLCANO
Kubernetes Native Batch System
KAI Scheduler
设备
NVIDIA
Ascend
AMD

THANK YOU